

React

Professional Complete Guide

Hooks · Patterns · Performance · Router · State · Testing · Custom Hooks · SSR

Book 6 of 15 · W3Schools Professional Series · 2026

Table of Contents

#	Chapter	Key Topics
1	JSX & Rendering	JSX rules, fragments, conditional rendering, lists
2	Components & Props	Props validation, composition, compound components
3	useState & Forms	State patterns, controlled forms, multi-step forms
4	useEffect & Data Fetching	Effect patterns, cleanup, data fetching, SWR-style
5	Advanced Hooks	useContext, useRef, useReducer, useMemo, useCallback
6	Custom Hooks	Building reusable hooks, patterns, recipes
7	Performance	React.memo, lazy/Suspense, profiler, code splitting
8	React Router v6	Routes, params, nested, protected, navigation
9	State Management	Context API, Zustand-style, Redux patterns
10	Portals, Error Boundaries & More	Portal, error boundary, concurrent features
11	Testing	React Testing Library, jest, user events, mocking
12	Exercises	20 exercises covering all React topics

Chapter 1: JSX & Rendering

JSX rules, fragments, lists, conditional patterns

JSX Rules & Gotchas

```
// JSX differences from HTML
// 1. className instead of class
// 2. htmlFor instead of for
// 3. camelCase events: onClick, onChange, onSubmit
// 4. Self-closing tags required: <br />, <img />, <input />
// 5. style is an object: style={{ color: "red", fontSize: 16 }}
// 6. Expressions in {}: {2 + 2}, {name}, {fn()}

// Fragments – avoid extra DOM nodes
function List({ items }) {
  return (
    <> /* shorthand for <React.Fragment> */
    <h2>Items</h2>
    <ul>
      {items.map((item, i) => (
        <React.Fragment key={item.id}>
        <li>{item.name}</li>
        <li>{item.desc}</li>
      </React.Fragment>
      ))}
    </ul>
    </>
  );
}

// Conditional rendering patterns
function Alert({ type, message, count }) {
  // Pattern 1: && (beware: 0 renders as "0"!)
  // Bad: {count && <Badge />} – renders "0" when count=0
  // Good: {count > 0 && <Badge count={count} />}

  // Pattern 2: Ternary
  const icon = type === "error" ? <ErrorIcon /> : <InfoIcon />;

  // Pattern 3: if/else before return
  if (!message) return null;

  // Pattern 4: switch / object map
  const colors = { error: "red", success: "green", info: "blue" };

  return <div style={{ color: colors[type] }}>{message}</div>;
}
```

Keys & List Rendering

```
// Keys must be unique among siblings and stable
// Bad: using array index (causes bugs with add/remove/reorder)
{items.map((item, i) => <li key={i}>{item}</li>)}

// Good: use stable unique IDs
{items.map(item => <li key={item.id}>{item.name}</li>)}

// Rendering tables
function DataTable({ columns, rows }) {
  return (
    <table>
      <thead>
        <tr>
          {columns.map(col => (
            <th key={col.key}>{col.label}</th>
          ))}
        </tr>
      </thead>
      <tbody>
        {rows.map(row => (
          <tr key={row.id}>
            {columns.map(col => (
              <td key={col.key}>{row[col.key]}</td>
            ))}
          </tr>
        ))}
      </tbody>
    </table>
  );
}
```

Chapter 2: Hooks Deep Dive

useEffect patterns, useReducer, useMemo, useCallback

useEffect Patterns

```
import { useEffect, useRef, useState } from "react";

// Pattern 1: Run once on mount
useEffect(() => {
  fetchData();
}, []); // empty deps = mount only

// Pattern 2: Run when deps change
useEffect(() => {
  document.title = `Page: ${page}`;
}, [page]);

// Pattern 3: Cleanup
useEffect(() => {
  const subscription = subscribe(id, handler);
  return () => subscription.unsubscribe(); // cleanup
}, [id]);

// Pattern 4: Abort fetch on unmount
useEffect(() => {
  const controller = new AbortController();
  fetch(`/api/users/${id}`, { signal: controller.signal })
    .then(r => r.json())
    .then(setUser)
    .catch(e => { if (e.name !== "AbortError") setError(e); });
  return () => controller.abort();
}, [id]);
```

useReducer, useMemo, useCallback

```

// useReducer for complex state
const initialState = {
  items: [], loading: false, error: null, filter: "all"
};

function reducer(state, action) {
  switch (action.type) {
    case "FETCH_START": return { ...state, loading: true, error: null };
    case "FETCH_OK": return { ...state, loading: false, items: action.payload };
    case "FETCH_ERROR": return { ...state, loading: false, error: action.error };
    case "SET_FILTER": return { ...state, filter: action.filter };
    default: throw new Error("Unknown action: " + action.type);
  }
}

const [state, dispatch] = useReducer(reducer, initialState);

// useMemo – expensive computation
const sortedItems = useMemo(() => {
  return [...items].sort((a, b) => a.name.localeCompare(b.name));
}, [items]); // only recompute when items changes

// useCallback – stable function reference
const handleDelete = useCallback((id) => {
  dispatch({ type: "DELETE", payload: id });
}, [dispatch]); // dispatch is stable, so this is also stable

// useRef – DOM access and mutable values without re-render
const inputRef = useRef(null);
const countRef = useRef(0); // like instance variable

useEffect(() => { inputRef.current?.focus(); }, []);

// Previous value pattern
function usePrevious(value) {
  const ref = useRef();
  useEffect(() => { ref.current = value; });
  return ref.current;
}

```

Chapter 3: Custom Hooks

Building reusable logic hooks

Custom Hook Recipes

useFetch hook

```
// useFetch – data fetching with loading/error
function useFetch(url) {
  const [state, setState] = useState({
    data: null, loading: true, error: null
  });
  useEffect(() => {
    let cancelled = false;
    fetch(url)
      .then(r => r.json())
      .then(data => { if (!cancelled) setState({ data, loading:false, error:null }); })
      .catch(error=> { if (!cancelled) setState({ data:null, loading:false, error }); });
    return () => { cancelled = true; };
  }, [url]);
  return state;
}
```

useLocalStorage, useDebounce, useMediaQuery

```

// useLocalStorage – persist state to localStorage
function useLocalStorage(key, initialValue) {
  const [value, setValue] = useState(() => {
    try { const i = localStorage.getItem(key);
    return i ? JSON.parse(i) : initialValue; }
    catch { return initialValue; }
  });
  const setStored = useCallback((val) => {
    const v = val instanceof Function ? val(value) : val;
    setValue(v);
    localStorage.setItem(key, JSON.stringify(v));
  }, [key, value]);
  return [value, setStored];
}

// useDebounce
function useDebounce(value, delay = 300) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const t = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(t);
  }, [value, delay]);
  return debounced;
}

// useMediaQuery
function useMediaQuery(query) {
  const [matches, setMatches] = useState(false);
  useEffect(() => {
    const mq = window.matchMedia(query);
    setMatches(mq.matches);
    const h = (e) => setMatches(e.matches);
    mq.addEventListener("change", h);
    return () => mq.removeEventListener("change", h);
  }, [query]);
  return matches;
}

```

Chapter 4: Performance & Testing

React.memo, lazy, Suspense, RTL

Performance Optimization

```
// React.memo – skip re-render if props unchanged
const ExpensiveChild = React.memo(function({ data, onAction }) {
  return <div>{/* expensive render */</div>;
}, (prevProps, nextProps) => {
  // custom comparison – return true to SKIP re-render
  return prevProps.data.id === nextProps.data.id;
});

// Code splitting – lazy load components
const Dashboard = React.lazy(() => import("./Dashboard"));
const Settings = React.lazy(() => import("./Settings"));

function App() {
  return (
    <Suspense fallback=<LoadingSpinner />>
    <Routes>
    <Route path="/dashboard" element=<Dashboard /> />
    <Route path="/settings" element=<Settings /> />
    </Routes>
  </Suspense>
  );
}

// Virtualization for long lists (react-virtual / react-window)
// Only render visible items
import { FixedSizeList } from "react-window";

function VirtualList({ items }) {
  const Row = ({ index, style }) => (
    <div style={style}>{items[index].name}</div>
  );
  return (
    <FixedSizeList height={600} itemCount={items.length} itemSize={50}>
    {Row}
    </FixedSizeList>
  );
}
```

Testing with React Testing Library


```

import { render, screen, fireEvent, waitFor } from "@testing-library/react";
import userEvent from "@testing-library/user-event";

// Component to test
function LoginForm({ onLogin }) {
  const [email, setEmail] = useState("");
  const [pass, setPass] = useState("");
  return (
    <form onSubmit={(e) => { e.preventDefault(); onLogin(email,pass); }}>
    <input placeholder="Email" value={email} onChange={e=>setEmail(e.target.value)} />
    <input placeholder="Password" value={pass} onChange={e=>setPass(e.target.value)}
    type="password" />
    <button type="submit">Login</button>
    </form>
  );
}

// Tests
describe("LoginForm", () => {
  it("calls onLogin with credentials when submitted", async () => {
    const user = userEvent.setup();
    const onLogin = jest.fn();
    render(<LoginForm onLogin={onLogin} />);

    await user.type(screen.getByPlaceholderText("Email"), "alice@example.com");
    await user.type(screen.getByPlaceholderText("Password"), "secret123");
    await user.click(screen.getByRole("button", { name: /login/i }));

    expect(onLogin).toHaveBeenCalledWith("alice@example.com", "secret123");
  });

  it("renders email and password fields", () => {
    render(<LoginForm onLogin={() => {}} />);
    expect(screen.getByPlaceholderText("Email")).toBeInTheDocument();
    expect(screen.getByPlaceholderText("Password")).toBeInTheDocument();
  });
});

```

Chapter 5: Exercises

20 professional React exercises

Exercises

FILL

Ex 1. What is the purpose of the key prop in lists?

■ **Answer:** Keys help React identify which items changed, were added, or removed. They must be unique among siblings and stable (not array index). Without proper keys, React may re-render incorrectly.

WRITE

Ex 2. Write a useFetch hook that handles loading, data, and error states.

■ **Answer:**

```
function useFetch(url){const[s,setS]=useState({data:null,loading:true,error:null});useEffect(()=>{let ok=true;fetch(url).then(r=>r.json()).then(d=>{if(ok)setS({data:d,loading:false,error:null});}).catch(e=>{if(ok)setS({data:null,loading:false,error:e});});return()=>{ok=false;};},[url]);return s;}
```

TRUEFALSE

Ex 3. useEffect with an empty dependency array [] runs after every render. (True/False)

■ **Answer:** FALSE – [] means run once after the initial mount only. No deps array means run after every render.

WRITE

Ex 4. Write a controlled search input with debouncing that calls an API after 500ms of no typing.

■ **Answer:**

```
function Search(){const[q,setQ]=useState('');const debouncedQ=useDebounce(q,500);useEffect(()=>{if(debouncedQ)fetchResults(debouncedQ);},[debouncedQ]);return<input value={q} onChange={e=>setQ(e.target.value)} placeholder='Search...'/>; }
```

EXPLAIN

Ex 5. When should you use useReducer instead of useState?

■ **Answer:** Use useReducer when: (1) next state depends on previous state in complex ways, (2) multiple pieces of state are related and change together, (3) state transitions are complex/need to be documented, (4) you want Redux-like patterns without a library.

WRITE

Ex 6. Write a useToggle custom hook.

■ **Answer:**

```
function useToggle(initial=false){const[state,setState]=useState(initial);const toggle=useCallback(()=>setState(s=>!s),[]);return[state,toggle];}
```

FILL

Ex 7. What does `React.memo` do and when should you use it?

■ **Answer:** `React.memo` is a HOC that memoizes a component – skips re-render if props haven't changed (shallow comparison). Use when: component renders often with same props, rendering is expensive. Don't use for every component.

WRITE

Ex 8. Write an `ErrorBoundary` class component.

■ **Answer:** `class ErrorBoundary extends React.Component`{state={hasError:false,error:null};static getDerivedStateFromError(e){return{hasError:true,error:e};}componentDidCatch(e,info){console.error(e,info);}render(){if(this.state.hasError)return<h2>Something went wrong</h2>;return this.props.children;}}

EXPLAIN

Ex 9. What is the difference between controlled and uncontrolled components?

■ **Answer:** Controlled: React state drives the input value (value + onChange). React is single source of truth. Uncontrolled: DOM manages its own state, accessed via ref. Use controlled for validation/formatting; uncontrolled for simple forms or integrating non-React code.

WRITE

Ex 10. Write a `useEventListener` hook.

■ **Answer:** `function useEventListener(event,handler,element=window){const savedHandler=useRef(handler);useEffect(()=>{savedHandler.current=handler;},[handler]);useEffect(()=>{const el=element?.current??element;const fn=e=>savedHandler.current(e);el.addEventListener(event,fn);return()=>el.removeEventListener(event,fn);},[event,element]);}`

FILL

Ex 11. What hook prevents creating a new function on every render in React?

■ **Answer:** `useCallback` – memoizes the function reference, only creating a new one when dependencies change. Useful when passing callbacks to memoized child components.

WRITE

Ex 12. Write a `Context` + custom hook for theme management.

■ **Answer:** `const ThemeCtx=createContext(null);export function ThemeProvider({children}){const[theme,setTheme]=useState('light');return<ThemeCtx.Provider value={{theme,setTheme}}>{children}</ThemeCtx.Provider>;}export const useTheme=()=>{const ctx=useContext(ThemeCtx);if(!ctx)throw new Error('useTheme must be inside ThemeProvider');return ctx;};`

EXPLAIN

Ex 13. What is prop drilling and how do you solve it?

■ **Answer:** Prop drilling: passing props through many component layers that don't need them, just to reach a deeply nested child. Solutions: (1) Context API for global state, (2) Component composition, (3) State management library (Zustand, Redux), (4) Component slots pattern.

WRITE

Ex 14. Write a Portal component for rendering modals outside the main DOM tree.

■ **Answer:**

```
import {createPortal} from 'react-dom'; function Modal({isOpen, onClose, children}) { if (!isOpen) return null; return createPortal( <div className='overlay' onClick={onClose}> <div className='modal' onClick={e=>e.stopPropagation()}> {children} <button onClick={onClose}>Close</button> </div> </div>, document.getElementById('portal-root') ); }
```

FILL

Ex 15. What does React.lazy() do?

■ **Answer:** React.lazy() enables code splitting by dynamically importing a component. The component is only loaded when rendered. Must be wrapped in Suspense with a fallback. Only works with default exports.

WRITE

Ex 16. Write a form with React Hook Form-style validation (manual implementation).

■ **Answer:**

```
function useForm(rules) { const [values, setVals] = useState({}); const [errors, setErrs] = useState({}); const register = name => ({value: values[name] || '', onChange: e => setVals(v => ({...v, [name]: e.target.value})); const validate = () => { const e = {}; Object.entries(rules).forEach(([k, r]) => { if (r.required && !values[k]) e[k] = 'Required'; }); setErrs(e); return Object.keys(e).length === 0; }); return {register, errors, validate, values}; }
```

EXPLAIN

Ex 17. What are React Server Components and how do they differ from Client Components?

■ **Answer:** Server Components render on the server, can access databases/files directly, have zero JS bundle impact on client, cannot use state/effects/browser APIs. Client Components (use client directive) run in browser, can use hooks and events. Default in Next.js 13+ App Router is server; add 'use client' for interactivity.

WRITE

Ex 18. Write a useAsync hook that handles an async function with status tracking.

■ **Answer:**

```
function useAsync(asyncFn) { const [status, setStatus] = useState('idle'); const [data, setData] = useState(null); const [error, setError] = useState(null); const run = useCallback(async (...args) => { setStatus('loading'); try { const d = await asyncFn(...args); setData(d); setStatus('success'); } catch (e) { setError(e); setStatus('error'); } }, [asyncFn]); return {run, data, error, status, isLoading: status === 'loading'}; }
```

FILL

Ex 19. What is the used hook used for in React 18+?

■ **Answer:** `useId` generates a stable, unique ID that persists across server and client renders – safe for SSR hydration. Use for accessibility attributes like `htmlFor/id` pairs, `aria-describedby`, etc.

WRITE

Ex 20. Write a compound component pattern for a `Select/Option` component.

■ **Answer:**

```
const SelectCtx=createContext(null); function
Select({value,onChange,children}){return<SelectCtx.Provider value={{value,onChange}}><div
role='listbox'>{children}</div></SelectCtx.Provider>;} Select.Option=function
Option({value,children}){const ctx=useContext(SelectCtx);return<div role='option'
aria-selected={ctx.value===value} onClick={()=>ctx.onChange(value)}>{children}</div>;} //
Usage: <Select value={v} onChange={setV}><Select.Option
value='a'>A</Select.Option></Select>
```